IN620 – Create your own library

Outcomes

You will take an existing Arduino sketch and create a Class based on it. From this you will create a library that can be imported into the Arduino IDE

Knowledge required to complete this lab

- You have used and understand the Arduino IDE.
- Understanding of Classes/Object Orientation as implemented in any language.

What are Arduino Libraries?

Good resources on this can be tricky to find, but I think this explanation is a real gem:

> "A Library is a collection of instructions that are designed to **execute a specific task.**
>
> Good libraries describe the tasks it tries to solve by the **name of the library** itself, and the **names of the functions** associated to the library.
>
> This is because the library **should be intuitive to use**, and all names **should describe, in an efficient way, all questions regarding *what*, *when*, and *where***, but **should conceal the *how***.
>
> On the Arduino the libraries often serve as a **wrapper** around the Arduino API language to **enhance code *readability* and *user experience***."
>
> *https://playground.arduino.cc/Code/Library/*

The last paragraph on the role of a library as a wrapper is important, as libraries form an important part of the plug'n'play role that Arduinos are designed to fill (Arduino users are usually smart, just not IT Experts!). This is something to keep in mind, especially if you are producing software for a wider audience.

Because I.T. Students are "smarter than the average bear" we are going to dig into the structure of an Arduino library, and show you how to create your own.

Classes in C++

All Arduino libraries are instances of a Class. As the Arduino C language is derived from C++, the syntax for creating a class is a little different to what you're used to in C#.

```
class Foo
{
public:
  Foo();  // Constructor
  ~Foo();  // Destructor - not used in Arduino C
  void SayHi();
  void Giggle();

private:
// not used at the moment
};
```

The Class Properties and methods

```
// Constructor
Foo::Foo()
{
}

// Destructor, never used in Arduino C
Foo::~Foo()
{
}

//Public methods
void Foo::SayHi()
{
  Serial.println("Hi");
}

void Foo::Giggle()
{
  Serial.println("Giggle!");
}
```

Class de/constructor

Method definitions with scope resolution operator – "connects" the method to the class defined above

Let's unpack this…

Class names are `CamelCase`, **starting with a capital**. Functions and variables are `camelCase` **starting with a lower-case letter.** Datatypes and return types are normal (Using Arduino syntax of course!)

```
Class Foo
{
          public:
           Foo();  // Constructor
           void printHello();
           int var1;
           string var2;
          private:
          // Nothing here…
};
```

Class properties are by default **private**. However we can use **Access Modifiers** to specify this. The two we will use are **public** and **private**. **The constructor is always public**.

Methods are defined outside of the class definition, by using the **resolution operation** `::` to **connect method definitions** to the **method declaration inside the class**.

```
// Constructor
Foo::Foo()
{
}

// Destructor, never used
Foo::~Foo()
{
}

//Public methods
void Foo::SayHi()
{
  Serial.println("Hi");
}

void Foo::Giggle()
{
  Serial.println("Giggle!");
}
```

Remember to include the **constructor** for the class. You may be familiar with the concept of a **destructor** used for cleanup, but we will ignore that as Arduino C doesn't really have a nice solution for that.

Let's assume you've made a class called SerialMonitor. Let us also assume we want a method called parseSerialInput() that returns a character (char datatype). We would write it like so:

```
class SerialMonitor()
{
          public:                         // Public Access
          SerialMonitor();                // Constructor
          char parseSerialInput();        // Method
};
```

We will write out the methods underneath like this:

```
SerialMonitor:: SerialMonitor()
{
          //constructor
}

char SerialMonitor::parseSerialInput()
{
          // Do stuff here! returns a char.
}
```

Don’t forget if your method returns something, to include the return-type as well.

This seems bizzare if you’re used to writing your methods “inside” the class, but we will use this technique to create the library files.

Exercise 1 – Create a Hello World class.

You will create a simple class that has a method that when called, prints “Hello World” to the Serial Monitor in the Arduino IDE.

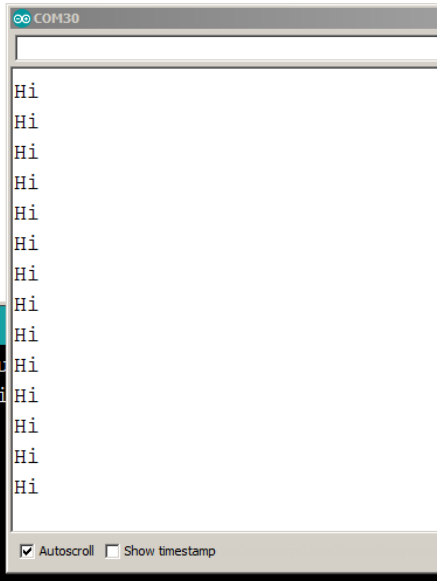
```
// Class template to use. Change the names!

Class Greeting
{
          public:
           Greeting ();  // Constructor
           void sayHi();

          private:
          // Nothing here
}; //Semicolon at the end of the class statement

// Constructor
Greeting:: Greeting()
{
} //NO semicolon

// Method
void Greeting:: sayHi()
{
   // code to print Hi using serial.println
}  //NO semicolon
```

Copy and paste this code skeleton above setup() in a new Sketch. Put this code above the **SetUp()** function in your sketch. This class uses the Serial monitor, so don’t forget to initialise the serial connection in Setup(). Create the code in the sayHi() method that will print “Hi” at the serial monitor.

Once you have done this, **create an instance of the class**. In loop, call the sayHi() method once per second:

```
Greeting welcome;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly:
  welcome.sayHi();
  delay(1000);
}
```

```
Done uploading.

Sketch uses 1624 bytes (5%) of program storage space. Maximu
Global variables use 190 bytes (9%) of dynamic memory, leavi
```



Exercise 2 – Blink!

In this exercise we will transform the Blink sketch into a Class. This will give you an example of how to take an existing sketch and adapt it into a library.

```
// the setup function runs once when you press reset or power the board
void setup() {
  // initialize digital pin LED_BUILTIN as an output.
  pinMode(LED_BUILTIN, OUTPUT);
}

// the loop function runs over and over again forever
void loop() {
  digitalWrite(LED_BUILTIN, HIGH);   // turn the LED on (HIGH is the voltage level)
  delay(1000);                       // wait for a second
  digitalWrite(LED_BUILTIN, LOW);    // turn the LED off by making the voltage LOW
  delay(1000);                       // wait for a second
}
```

In the **constructor** of the class, we will **initialise the digital pin.**

We will **create a blink method**, that will: blink on → 1 sec pause → blink off. We will call this method in loop().

Create an instance of your Blink class and call the blink method from inside Loop().

```
class Blinker
{
  public:
    Blinker();  // Constructor
    void blinkLED();
  }

Blinker::Blinker()
{
  // Something will go here...
  }

Blinker::blinkLED()
{
  // Code to blink on → 1 sec pause → blink off
  }

Blinker blinkme; // Create an instance of the class to use in Loop()
```

Exercise 3 – Passing in data to a class.

Let's look at creating a class where we can **pass an argument into a method**.

We will create a class with a method that will **take in an integer** and **return the square of that integer**.

Here is what our class will look like:

```
class BasicMaths
{
  public:
    BasicMaths();
    int returnSquare(int value);
  };
```

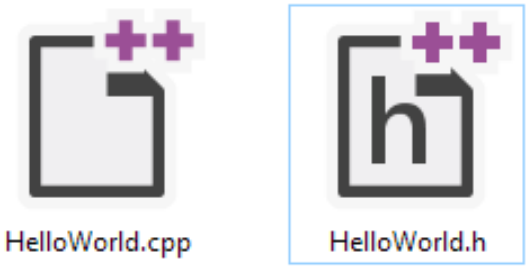
Create the method definitions with the resolution operation – one for the constructor and one for our returnSquare() method. The method returnSquare takes in an integer and returns an integer.

This exercise also illustrates what libraries do – they act as a user friendly wrapper for code.

Arduino Library Format

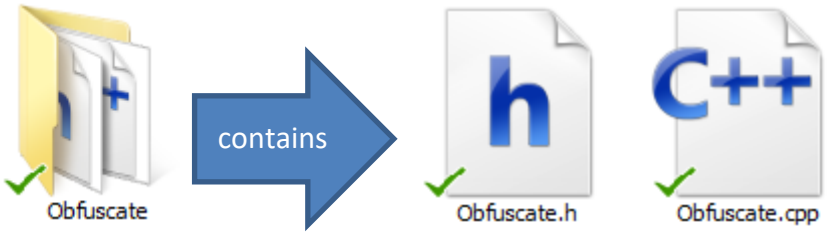
Now we get to the point of this – Arduino Libraries consist of two files:

A .cpp (dot-cpp) **source file**
A .h (dot-h) **header file**



The **header file** contains the **code that defines the class**, and the **source file** contains the **code for each method**.

Each file has the same name as the class we're using and should go into a folder that has the exact same name!



We also need to add some extra code to the source and header files so the Arduino IDE understands that this is a library.

Header file

```
#ifndef classname_h
#define classname_h
#include <Arduino.h>
ClassClass definition goes here
#endif
```

The header file begins with three lines of code that are designed to prevent issues if we attempt to import a library if it has already been included. **Classname_h** refers to the particular class we are using, so if our class was called **Randomiser**, we would type **Randomiser_h**. If it was called **Bob**, we would use **Bob_h**. Arduino.h is a library that must be included in *this* library we are creating.

```
#ifndef Greeting_h
#define Greeting_h

#include "Arduino.h"

class Greeting
{
  public:
    Greeting();
    void sayHi();

  private:
  // Nothing here
};
#endif
```

Source File

#include "classname.h"
method definitions go here

This file has a one-line header **Classname.h** that refers to the **header file you've created**. Don't forget to put the constructor in this file – even if it is empty.

```
#include "Greeting.h"

Greeting::Greeting()
{
}

void Greeting::sayHi()
{
  Serial.println("Hi");
}
```

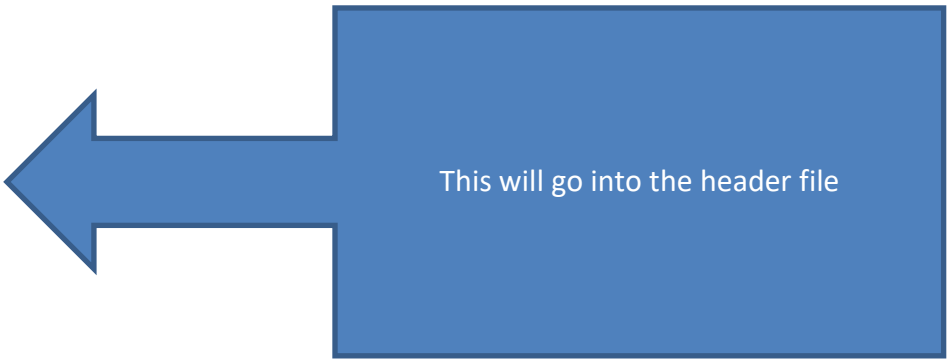
I'm hoping you can see the way we have split up a Class across the two files (The code that defines the class, and the method implementations)

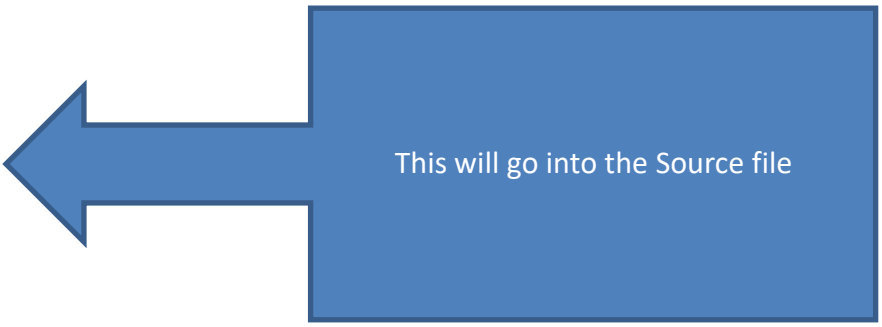
You will want to use a plain text editor to create the library, so copy and paste the code from the Arduino IDE into something like notepad++ etc. Be careful of copying quotemarks, encoding etc.

Exercise – Adapt code into a Library

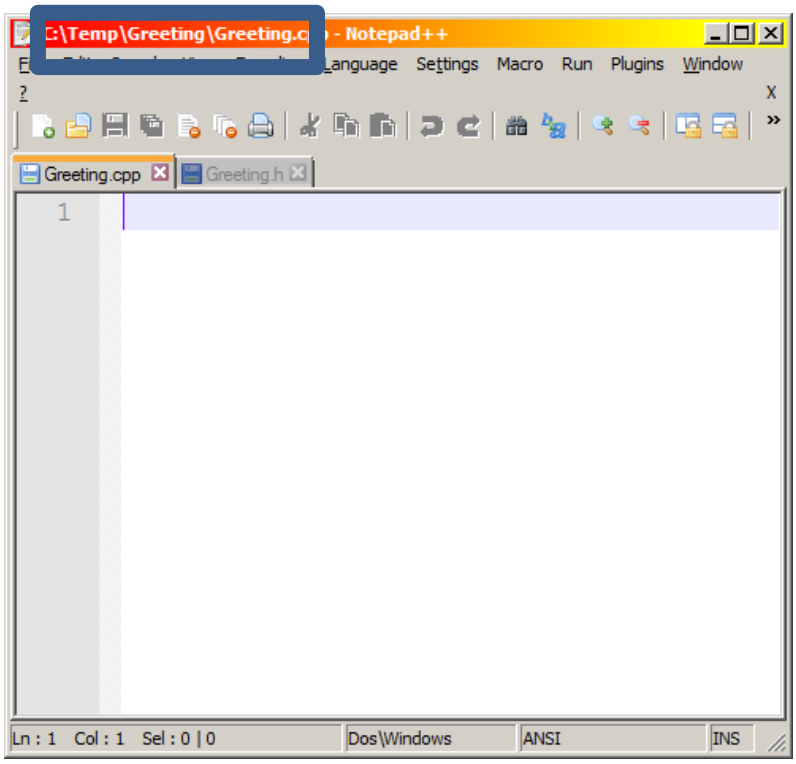
Take our simple hello world example:

```
class Greeting
{
  public:
    Greeting();
    void sayHi();

  private:
  // Nothing here
};

Greeting::Greeting()
{
}

void Greeting::sayHi()
{
  Serial.println("Hi");
}
```

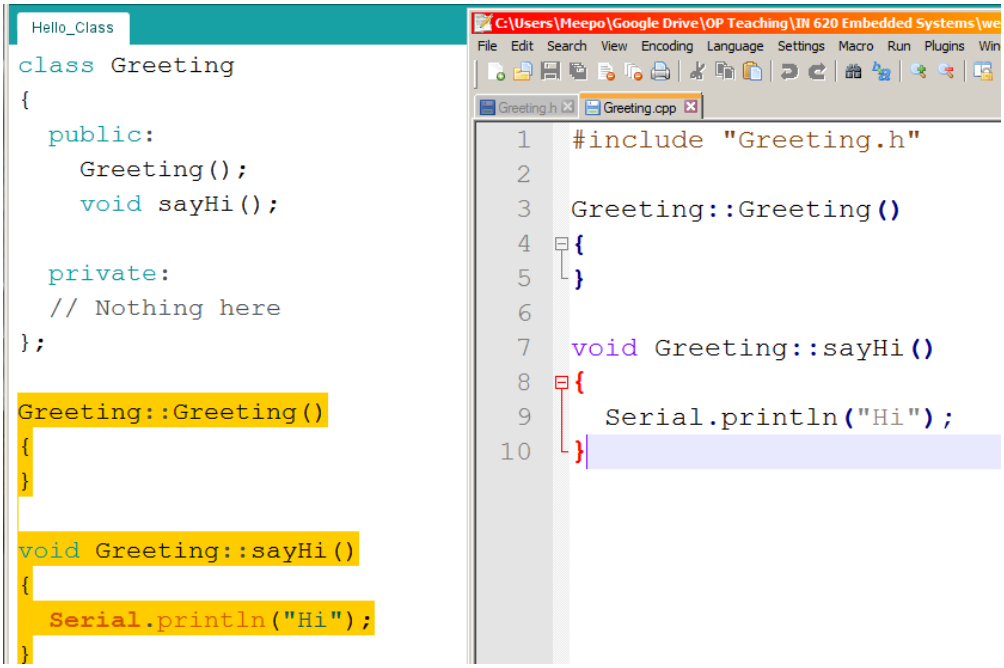




Create a folder called Greeting. Create two blank text files, Greeting.cpp and Greeting.h

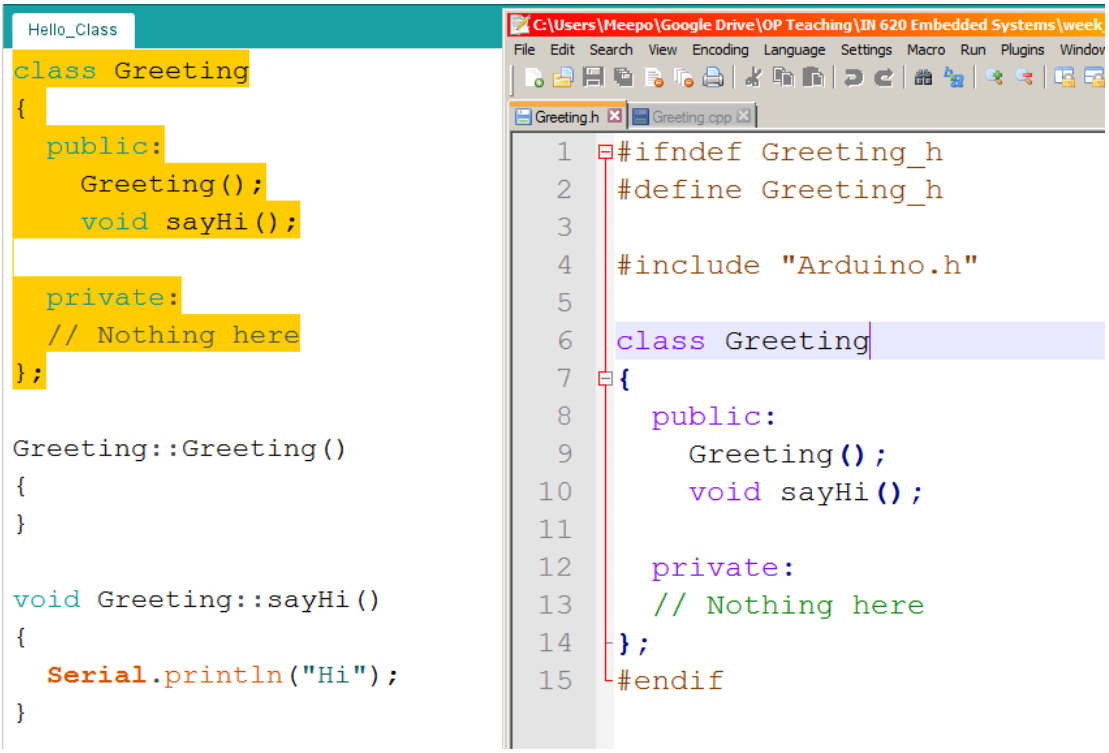


I'm going to start with the **source** file. **Add the file header and Methods**



Save this file!

Next, **create the header file**. **Copy and paste** the **class** into this file



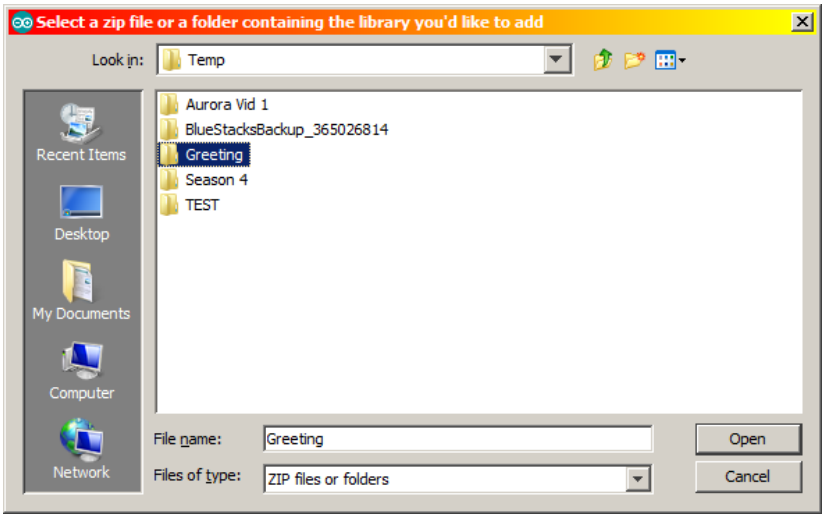
Don't forget to save!

You will now have a **folder called Greeting** that should contain **two files**, **Greeting.h** and **Greeting.cpp**

Testing the library!

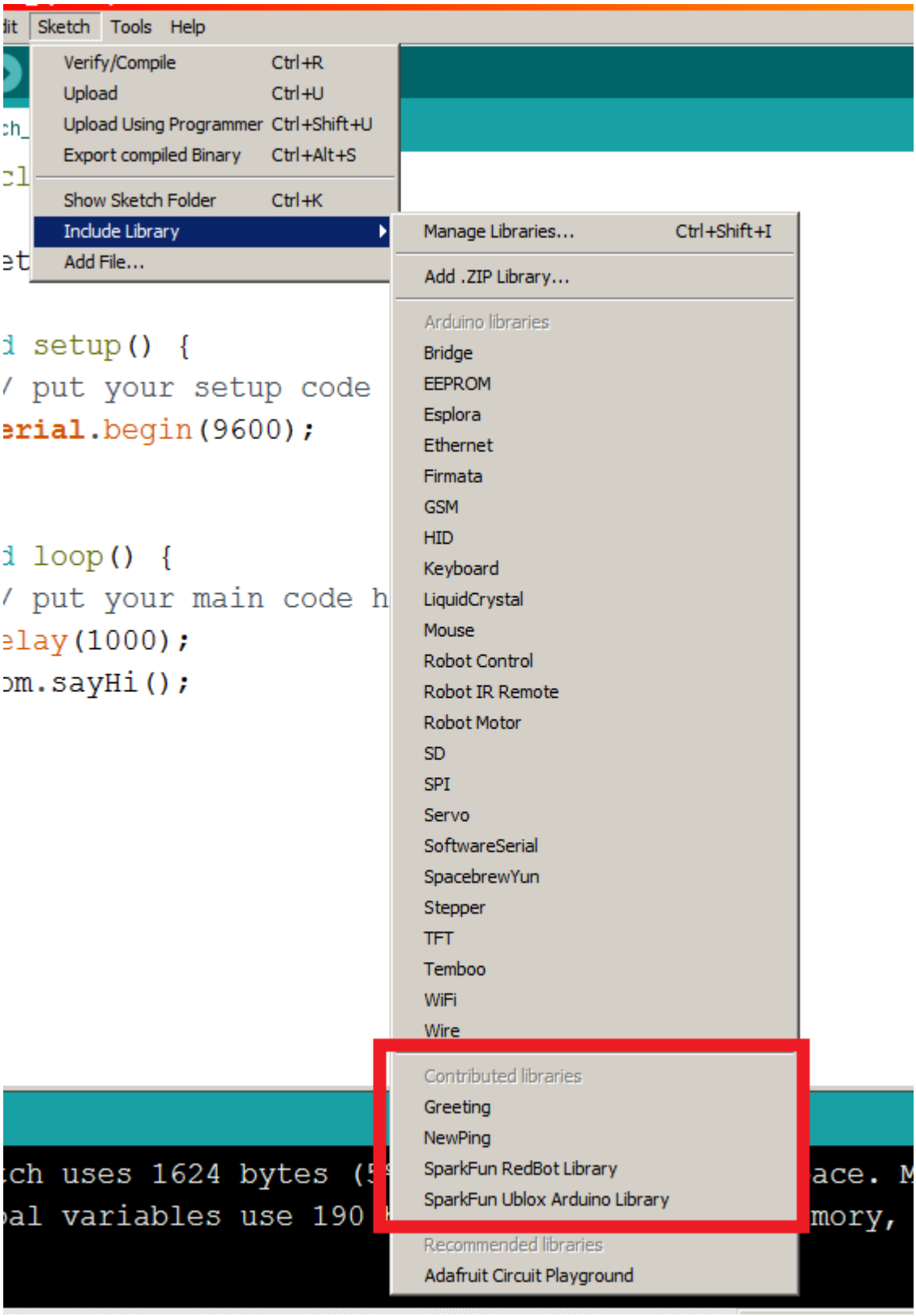
Use the library manager in the Arduino IDE to browse to your new library:

sketch menu → include library → add .ZIP Library



Just click on the folder you created, and click on open (ie, don't go into the folders and click on the CPP or H file).

Include the library in the usual way, using the library manager:



You should see it in the menu of contributed libraries. Create an instance of the included library and access the sayHi() method:

sketch_apr27c

```
#include <Greeting.h>

Greeting tom;

void setup() {
  // put your setup code here, to run once:
  Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly:
  delay(1000);
  tom.sayHi();
}
```

When Things Go Wrong!

This can be a slightly fraught process. Make sure you check that:

- Your original Class works before you attempt to convert it into a library.
- Folder, header file, and source files, class name, all have the same name. Same spelling, capitalisation etc.
- Be careful of copying and pasting text between editors – make sure that quote marks come across as plain quote marks.
- Check spelling of your methods. The Arduino IDE does not have code-completion/intellisense.

The Arduino IDE stores imported libraries in the **Arduino folder** stored in your **user profile** (Windows) or in the **Arduino folder** (Portable version). If you have made a mistake in your code, you will need to delete this imported library (just deleted the folder) before you attempt to reimport any debugged code.

Exercise 4 – extension.

Customise the greeting library. We want to be able to say “hello” in other ways. Add two more methods to your original greeting Class, and create a new greeting library from this.

Exercise 5 – ROT13 Obfuscation

We will utilise the sketch "Rot13" to create a simple obfuscation library. Obfuscation is a technique to superficially scramble data before transmitting it or applying more heavy duty techniques. Although it is the least secure kind of encryption, it is a handy way to hide information from casual prying.

Rot13 is a form of Caesar cypher – a simple transposition cypher. This has the advantage that the same algorithm that encrypts the message will also decrypt the message.

The sketch "Rot13" has only two methods – the first method converts a string into an array of characters. The second method takes a single character and finds the transcribed value

The example sketch is just a plain sketch, there's no classes or anthing in there. So a suggested workflow for you is:

1. Create a class that will hold the methods used in the sketch.
2. **Test this class inside a sketch**, to see that it works. **Fix bugs at this stage.**
3. Final step is to create the header and source files for the library.

So, begin with the sketch, create a Class:

```
Class Obfuscation{
     public:
     // What public methods and properties do we need?

     private:
     // you probably won't have any private properties here
}
```